

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Laboratory Report

Experiment 1

Implementation of a Single Layer Perceptron for Binary Classification

Name: T. A. Prasanna

Roll No. 24110124

1 Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. The experiment covers the perceptron learning algorithm, the role of activation functions, visualization of the learning process, and evaluation of the classifier using a real-world dataset (the Banknote Authentication dataset).

2 Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3 Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron used in this experiment employs only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions are studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4 Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link: <https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

5 Experimental Procedure

Task 1: Dataset Exploration

- Loaded the dataset
- Displayed the first five samples.
- Determined dataset dimensions.
- Identified missing values.
- Displayed descriptive statistics.

Task 2: Exploratory Data Analysis

Generated the following: Histograms, Correlation Heatmap, Scatter Plot, and Boxplots.

Task 3: Data Preprocessing

- Normalized all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implemented from scratch:

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Displayed: Epoch Number, Misclassified Samples, Updated Weights, Updated Bias.

Task 6: Model Evaluation

Computed: Accuracy, Precision, Recall, F1-score, and Confusion Matrix.

6 Source Code

The complete implementation of the Single Layer Perceptron, developed from scratch in Python, is provided below.

Listing 1: Single Layer Perceptron – Implementation

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 import urllib.request
6 from sklearn.model_selection import train_test_split
7 from sklearn.preprocessing import StandardScaler
8 from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
9                               confusion_matrix, ConfusionMatrixDisplay)
10 from sklearn.linear_model import Perceptron as SklearnPerceptron
11
12 np.random.seed(42)
13 plt.rcParams["figure.figsize"] = (7, 5)
14 DATA_URL = ("https://archive.ics.uci.edu/ml/machine-learning-databases/"
15             "00267/data_banknote_authentication.txt")
16
17 LOCAL_FILE = "data_banknote_authentication.csv"
18 COLUMNS = ["Variance", "Skewness", "Curtosis", "Entropy", "Class"]
19 def dataset():
20     try:
21         urllib.request.urlretrieve(DATA_URL, LOCAL_FILE)
22         print("Dataset downloaded from UCI repository.")
23     except Exception as e:
24         print(f"Could not download dataset automatically ({e}). "
```

```

25         f"Looking for a local copy: {LOCAL_FILE}")
26     df = pd.read_csv(LOCAL_FILE, header=None, names=COLUMNS)
27     return df
28 df = dataset()
29
30
31 #Task 1
32 print("\nFirst five samples:\n", df.head())
33 print("\nDataset dimensions:", df.shape)
34 print("\nMissing values per column:\n", df.isnull().sum())
35 print("\nDescriptive statistics:\n", df.describe())
36
37 #Task 2
38 df.drop(columns="Class").hist(bins=30, figsize=(10, 8))
39 plt.suptitle("Feature Histograms")
40 plt.tight_layout()
41 plt.savefig("histograms.eps", dpi=600)
42
43 plt.close()
44 plt.figure(figsize=(6, 5))
45 sns.heatmap(df.corr(), annot=True, cmap="coolwarm", fmt=".2f")
46 plt.title("Correlation Heatmap")
47
48 plt.tight_layout()
49 plt.savefig("correlation_heatmap.eps", dpi=600)
50 plt.close()
51 plt.figure()
52 sns.scatterplot(data=df, x="Variance", y="Skewness", hue="Class", palette="Set1")
53 plt.title("Scatter Plot: Variance vs Skewness by Class")
54 plt.tight_layout()
55
56 plt.savefig("scatter_plot.eps", dpi=600)
57 plt.close()
58 plt.figure(figsize=(10, 6))
59 df.drop(columns="Class").boxplot()
60 plt.title("Boxplots of Features")
61 plt.tight_layout()
62 plt.savefig("boxplots.eps", dpi=600)
63 plt.close()
64 print("EDA plots saved: histograms.eps, correlation_heatmap.eps, "
65       "scatter_plot.eps, boxplots.eps")
66
67 #Task 3
68 X = df.drop(columns="Class").values
69 y = df["Class"].values
70
71 scaler = StandardScaler()
72 scaled_x = scaler.fit_transform(X)
73
74 training_x, testing_x, training_y, testing_y = train_test_split(
75     scaled_x, y, test_size=0.20, random_state=42, stratify=y
76 )
77
78 print("Training set size:", training_x.shape)

```

```

79 print("Testing set size:", testing_x.shape)
80
81
82 class Perceptron:
83     def __init__(self, features, lr =0.01, epochs=50):
84         self.eta = lr
85         self.epochs = epochs
86         self.weights = np.zeros(features)
87         self.bias = 0.0
88         self.errorsPerepoch = []
89         self.weight_prev = []
90         self.bias_prev = []
91
92     @staticmethod
93     def step_activation(z):
94         return np.where(z >= 0, 1, 0)
95
96
97     def net_input(self, x):
98         return np.dot(x, self.weights) + self.bias
99
100     def predict(self, X):
101         return self.step_activation(self.net_input(X))
102
103
104     def fit(self, X, y, verbose=True):
105         for epoch in range(1, self.epochs + 1):
106             misclassified = 0
107             for xi, target in zip(X, y):
108                 z = self.net_input(xi)
109                 y_hat = self.step_activation(z)
110                 update = self.eta * (target - y_hat)
111                 if update != 0.0:
112
113                     self.weights += update * xi
114                     self.bias += update
115                     misclassified += 1
116
117             self.errorsPerepoch.append(misclassified)
118
119             self.weight_prev.append(self.weights.copy())
120             self.bias_prev.append(self.bias)
121
122             if verbose:
123                 print(f"Epoch {epoch:3d} | Misclassified: {misclassified:4d} | "
124                       f"Weights: {np.round(self.weights, 4)} | "
125                       f"Bias: {self.bias:.4f}")
126
127             if misclassified == 0:
128                 break
129         return self
130
131
132 #Task 5

```

```

133 LR = 0.01
134 EPOCHS = 50
135 model = Perceptron(features=training_x.shape[1], lr=LR,
136                     epochs=EPOCHS)
137 model.fit(training_x, training_y)
138
139 plt.figure()
140 plt.plot(range(1, len(model.errorsPerepoch) + 1), model.errorsPerepoch,
141          marker="o")
142 plt.xlabel("Epoch")
143 plt.ylabel("Misclassified Samples")
144 plt.title(f"Training Error vs Epoch (eta = {LR})")
145
146 plt.tight_layout()
147 plt.savefig("training_error_vs_epoch.eps", dpi=600)
148 plt.close()
149 weight_hist = np.array(model.weight_prev)
150 plt.figure()
151 for i in range(weight_hist.shape[1]):
152     plt.plot(weight_hist[:, i], label=f"w{i+1} ({COLUMNS[i]})")
153 plt.xlabel("Epoch")
154 plt.ylabel("Weight Value")
155 plt.title("Weight Evolution")
156
157 plt.legend()
158 plt.tight_layout()
159 plt.savefig("weight_evolution.eps", dpi=600)
160 plt.close()
161 plt.figure()
162
163 plt.plot(model.bias_prev, marker="o", color="darkorange")
164 plt.xlabel("Epoch")
165 plt.ylabel("Bias Value")
166
167 plt.title("Bias Evolution")
168 plt.tight_layout()
169 plt.savefig("bias_evolution.eps", dpi=600)
170 plt.close()
171 epoch_table = pd.DataFrame({
172     "Epoch": range(1, len(model.errorsPerepoch) + 1),
173     "Errors": model.errorsPerepoch,
174     "Weight_1": weight_hist[:, 0],
175     "Weight_2": weight_hist[:, 1],
176     "Bias": model.bias_prev,
177 })
178 epoch_table.to_csv("epoch_wise_learning.csv", index=False)
179 print("\nEpoch-wise learning table saved to epoch_wise_learning.csv")
180 print(epoch_table.head())
181
182 #Task 6
183 pred_y = model.predict(testing_x)
184 acc = accuracy_score(testing_y, pred_y)
185
186 prec = precision_score(testing_y, pred_y)

```

```

187 recall = recall_score(testing_y, pred_y)
188 f1 = f1_score(testing_y, pred_y)
189 cm = confusion_matrix(testing_y, pred_y)
190 print(f"Accuracy : {acc:.4f}")
191 print(f"Precision: {prec:.4f}")
192 print(f"Recall : {recall:.4f}")
193 print(f"F1-score : {f1:.4f}")
194 print("Confusion Matrix:\n", cm)
195
196 disp = ConfusionMatrixDisplay(confusion_matrix=cm,
197                               display_labels=["Authentic (0)", "Forged (1)"])
198 disp.plot(cmap="Blues")
199 plt.title("Confusion Matrix")
200
201 plt.tight_layout()
202 plt.savefig("confusion_matrix.eps", dpi=600)
203 plt.close()
204
205 summary = pd.DataFrame({
206     "Metric": ["Dataset Size", "Train/Test Split", "Learning Rate", "Epochs Run",
207               "Final Weights", "Final Bias", "Accuracy", "Precision",
208               "Recall", "F1-score"],
209     "Value": [df.shape[0], "80/20", LR, len(model.errorsPerepoch),
210               np.round(model.weights, 4).tolist(), round(model.bias, 4),
211               round(acc, 4), round(prec, 4),
212               round(recall, 4), round(f1, 4)]
213 })
214 summary.to_csv("training_summary.csv", index=False)
215 print("\nTraining summary saved to training_summary.csv")
216
217 print("\nLearning Rate Comparison")
218 lrs = [0.001, 0.01, 0.1]
219 styles = [
220     dict(lw=8, alpha=0.35, linestyle="-", color="tab:blue"),
221     dict(lw=5, alpha=0.55, linestyle="--", color="tab:orange"),
222     dict(lw=2, alpha=1.0, linestyle="-", color="tab:green"),
223 ]
224 plt.figure()
225 for lr, style in zip(lrs, styles):
226     m = Perceptron(features=training_x.shape[1], lr=lr, epochs=EPOCHS)
227     m.fit(training_x, training_y, verbose=False)
228
229     plt.plot(range(1, len(m.errorsPerepoch) + 1), m.errorsPerepoch,
230              marker="o", markersize=3, label=f"eta = {lr}", **style)
231
232     print(f"eta = {lr:<6} -> Epochs to converge: {len(m.errorsPerepoch):3d}, "
233           f"Final Weights: {np.round(m.weights, 4)}, Final Bias: {m.bias:.4f}")
234
235 plt.xlabel("Epoch")
236 plt.ylabel("Misclassified Samples")
237 plt.title("Learning Rate Comparison\n(curves overlap eta only rescales weights, not the
238           error trajectory)")
239 plt.legend()
240 plt.tight_layout()

```

```

240 plt.savefig("learning_rate_comparison.eps", dpi=600)
241 plt.close()
242
243 plt.xlabel("Epoch")
244 plt.ylabel("Misclassified Samples")
245 plt.title("Learning Rate Comparison")
246
247 plt.legend()
248 plt.tight_layout()
249 plt.savefig("learning_rate_comparison.eps", dpi=600)
250 plt.close()
251 print("\nOptional")
252 X2 = scaled_x[:, :2]
253 training_x2, testing_x2, training_y2, testing_y2 = train_test_split(
254     X2, y, test_size=0.20, random_state=42, stratify=y
255 )
256
257 model_2d = Perceptron(features=2, lr=0.01, epochs=50)
258 model_2d.fit(training_x2, training_y2, verbose=False)
259
260 x_min, x_max = X2[:, 0].min() - 1, X2[:, 0].max() + 1
261 y_min, y_max = X2[:, 1].min() - 1, X2[:, 1].max() + 1
262 xx, yy = np.meshgrid(np.linspace(x_min, x_max, 300),
263                     np.linspace(y_min, y_max, 300))
264 grid_preds = model_2d.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
265
266 plt.figure()
267 plt.contourf(xx, yy, grid_preds, alpha=0.3, cmap="coolwarm")
268 sns.scatterplot(x=X2[:, 0], y=X2[:, 1], hue=y, palette="Set1", edgecolor="k")
269
270
271 plt.xlabel("Variance (scaled)")
272 plt.ylabel("Skewness (scaled)")
273 plt.title("Decision Boundary (Variance vs Skewness)")
274 plt.tight_layout()
275 plt.savefig("decision_boundary.eps", dpi=600)
276 plt.close()
277
278 print("\nComparison with Scikit-learn's Perceptron")
279 sk_model = SklearnPerceptron(max_iter=EPOCHS, eta0=LR, random_state=42)
280 sk_model.fit(training_x, training_y)
281 sk_prediction = sk_model.predict(testing_x)
282
283 print("Scratch Perceptron -> Accuracy: {:.4f}, Precision: {:.4f}, Recall: {:.4f}, F1:
284     {:.4f}".format(
285         acc, prec, recall, f1))
286 print("Sklearn Perceptron -> Accuracy: {:.4f}, Precision: {:.4f}, Recall: {:.4f}, F1:
287     {:.4f}".format(
288         accuracy_score(testing_y, sk_prediction), precision_score(testing_y, sk_prediction),
289         recall_score(testing_y, sk_prediction), f1_score(testing_y, sk_prediction)))

```

7 Experimental Results

This section summarizes the quantitative outcomes obtained after training and evaluating the Single Layer Perceptron on the Banknote Authentication dataset.

Accuracy	0.9855
Precision	0.9683
Recall	1.0000
F1-score	0.9839

8 Generated Plots with Interpretation

Each plot below is accompanied by a brief interpretation (2–3 sentences) as required by the report guidelines.

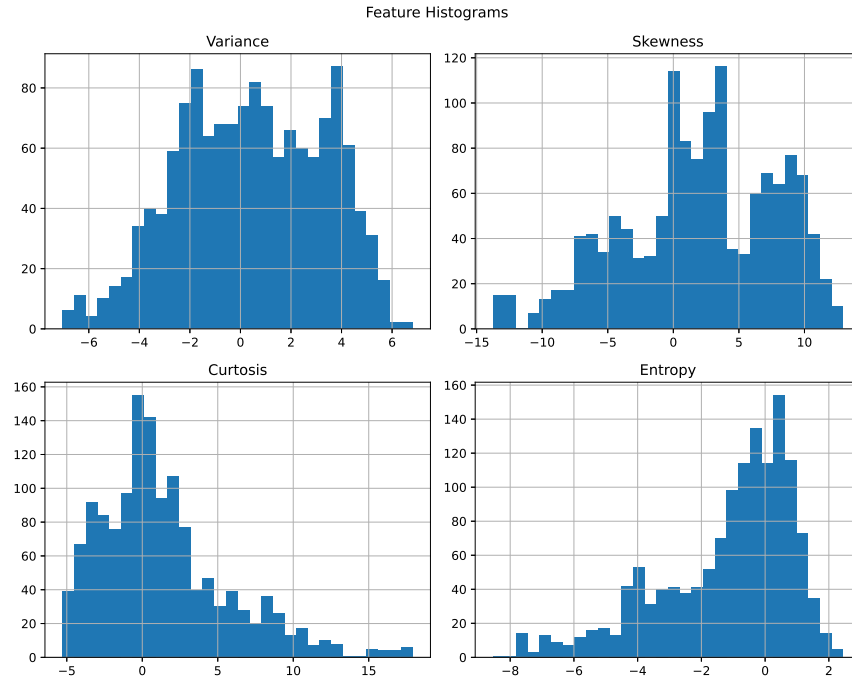


Figure 1: Feature Histograms

Interpretation: The variance graph is equally distributed while the others are skewed towards one side which indicates that the points can be separated into classes.

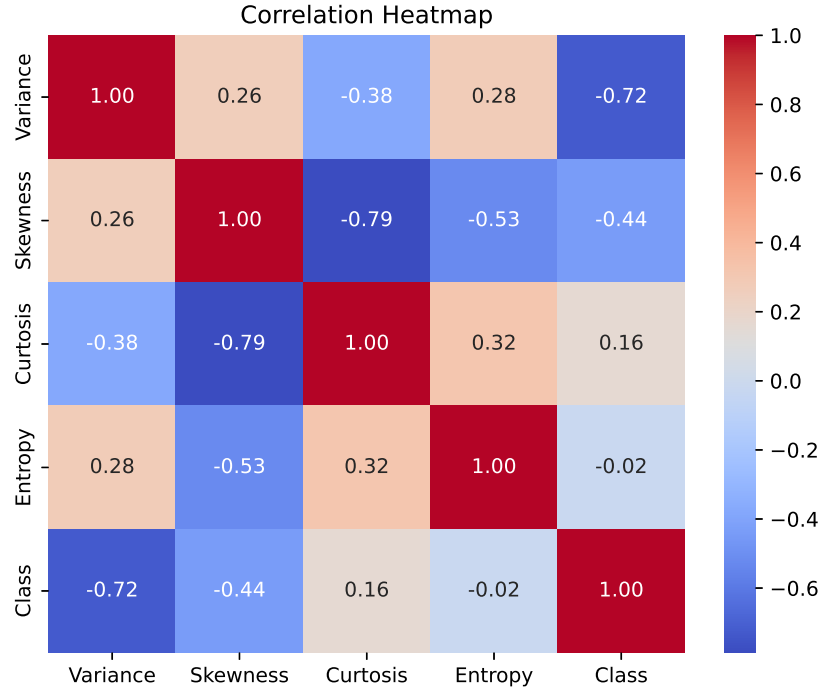


Figure 2: Correlation Heatmap

Interpretation: The attributes are not highly correlated and they can be used for training

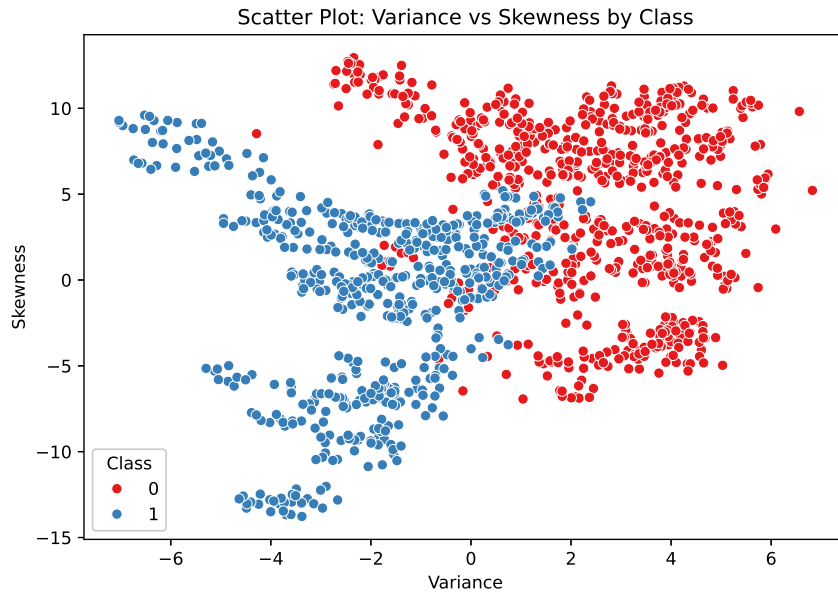


Figure 3: Scatter Plot of Class Distribution

Interpretation: The datapoints for the two features are overlapping with each other. Thus all features are required for proper classification.

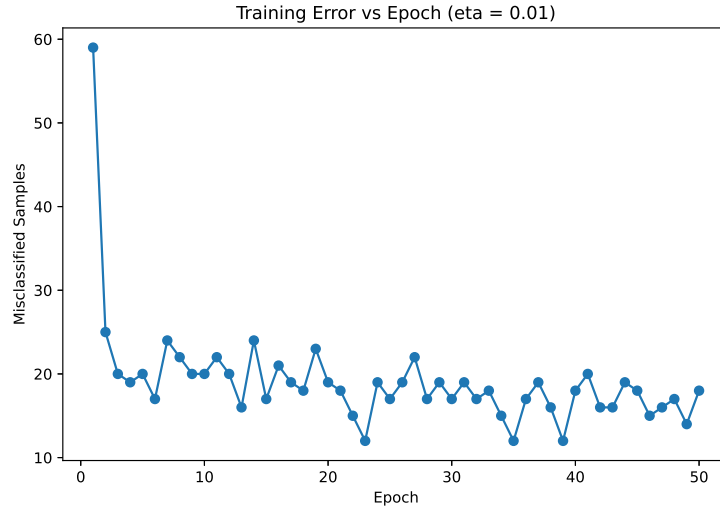


Figure 4: Training Error vs Epoch

Interpretation: The model's training error starts decreasing after few epochs. It starts converging after 3 epochs.

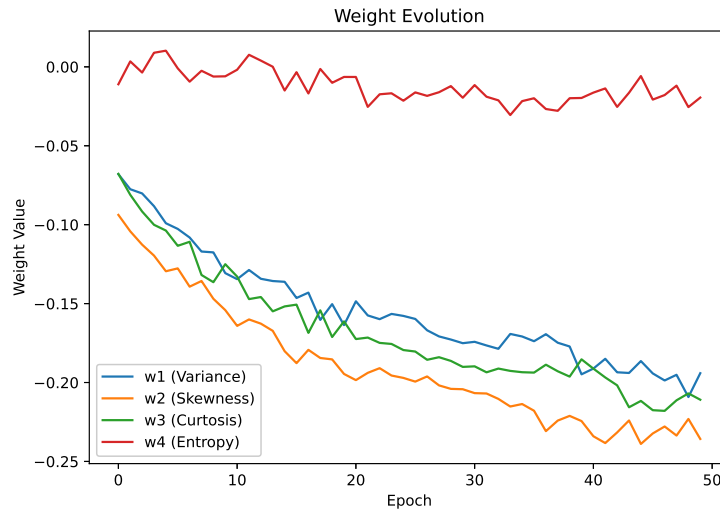


Figure 5: Weight Evolution over Epochs

Interpretation: The features start reducing with increasing epochs and the weights decreases.

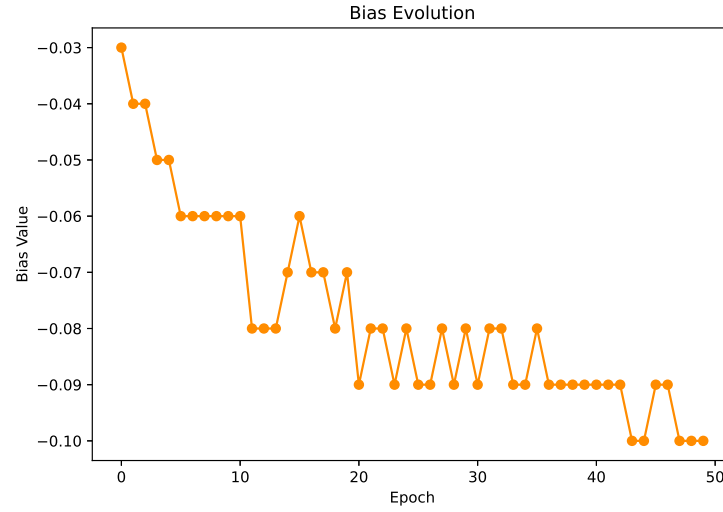


Figure 6: Bias Evolution over Epochs

Interpretation: The bias gradually reduces with the number of epochs showing good training and optimization.

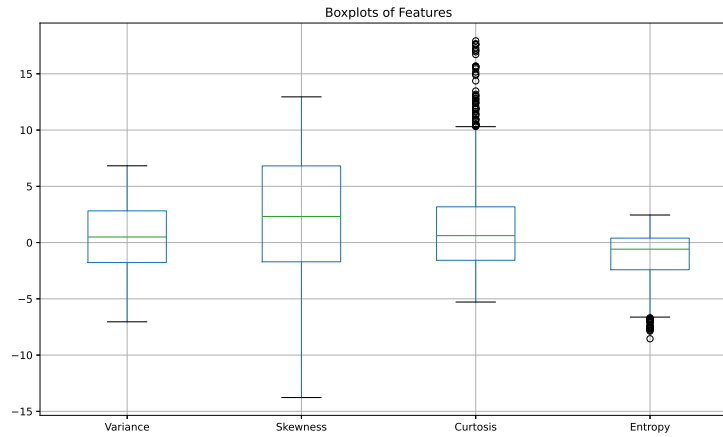


Figure 7: Decision Boundary

Interpretation: The Entropy and Kurtosis have outliers beyond whiskers, which means that skewness is presented in graphs.

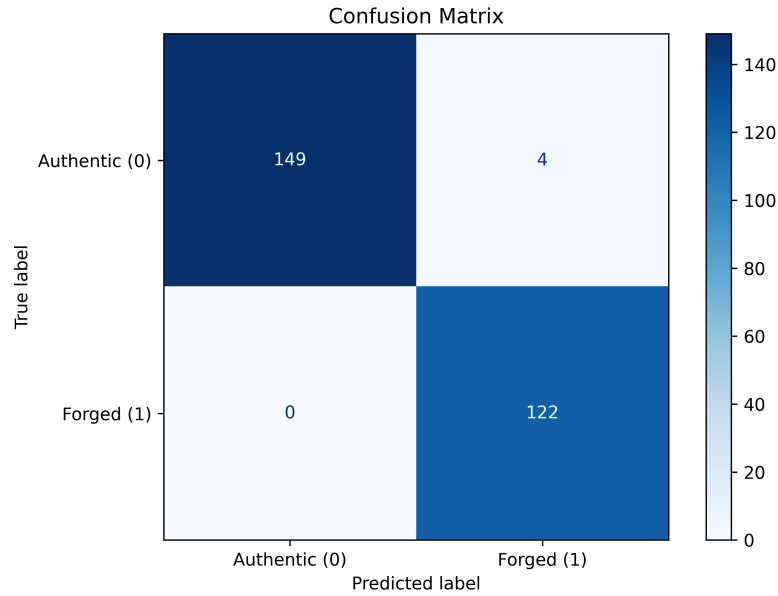


Figure 8: Confusion Matrix

Interpretation: Out of 256 samples, 149 are original, and 122 are duplicates, where 4 were mistakenly identified as duplicates, which confirms a high accuracy rate of the model.

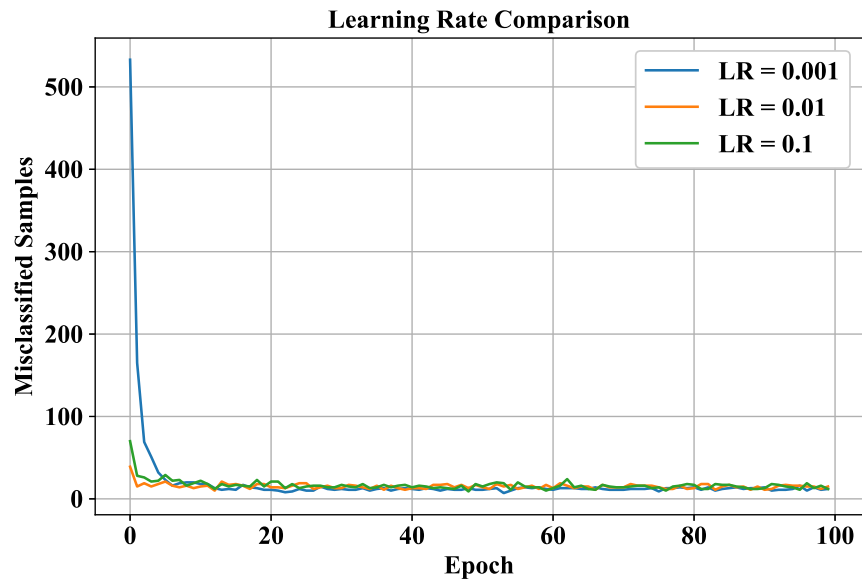


Figure 9: Learning Rate Comparison

Interpretation: The learning rates 0,01 and 0,1 demonstrate a stable convergence, while the number of misclassified samples is constantly decreasing for the learning rate 0,001 after 5 epochs.

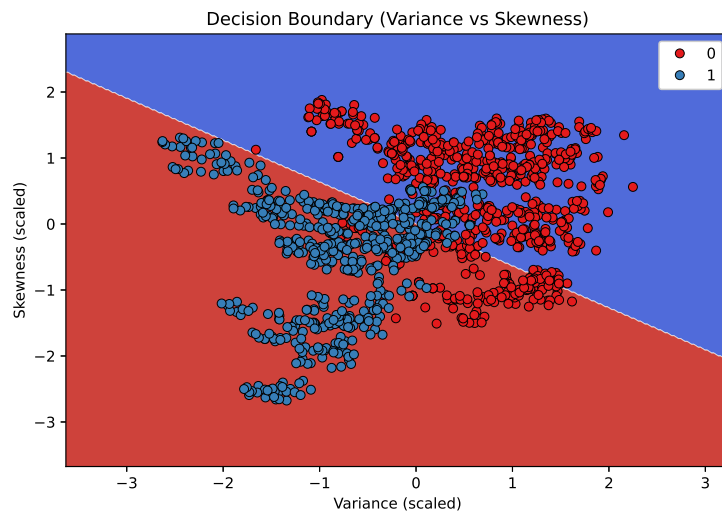


Figure 10: Decision Boundary (Optional – two selected features)

Interpretation: There are more overlapping points near the line, which suggests that not only two, but also all others variables contribute to classification.

9 Performance Tables

Training Summary

Dataset Size	1372
Train/Test Split	80 - 20
Learning Rate	0.01
Number of Epochs	50
Final Weights	(-0.1941, -0.2358, -0.2109, -0.0195)
Final Bias	-0.1000

Epoch-wise Learning

Epoch	Errors	Weight 1	Weight 2	Bias
1	59	-0.068025	-0.093809	-0.03
10	20	-0.1307	-0.1541	-0.06
20	19	-0.1636	-0.1946	-0.07
30	17	-0.1751	-0.2043	-0.08
40	18	-0.1948	-0.2244	-0.09
50	18	-0.1941	-0.2358	-0.10

10 Discussion

1. Step vs. Sigmoid Activation

The Step Activation Function produces binary output (0 or 1) while the Sigmoid function produces a continuous output in the range of (0,1). The Step activation function is not used in practice due to non differentiability at $z=0$. The Sigmoid activation on the other hand produces probabilistic outputs which are easy to understand and suitable for binary classification at the same time allows gradients to flow smoothly during backpropagation.

2. Comparison with Scikit-learn's Perceptron

Manual Perceptron -> Accuracy: 0.9855, Precision: 0.9683, Recall: 1.0000, F1: 0.9839

Sklearn Perceptron -> Accuracy: 0.9745, Precision: 0.9457, Recall: 1.0000, F1: 0.9721

3. Effect of Learning Rate (0.001, 0.01, 0.1)

Learning Rate: 0.001, Epochs: 50, Weights: [-0.0194 -0.0236], Bias: -0.0100

Learning Rate: 0.01, Epochs: 50, Weights: [-0.1941 -0.2358], Bias: -0.1000

Learning Rate: 0.1, Epochs: 50, Weights: [-1.941 -2.3575], Bias: -1.0000

A very small learning rate results in slow but steady convergence requiring more epochs to minimize the error. On the contrary, A learning rate too big can cause weight divergence while a moderate learning rate usually results in the fastest convergence.

4. Why a Single Layer Perceptron Cannot Solve XOR

XOR problem is not linearly separable, which means that there is no linear classifier that can separate the two classes, which would be represented by a straight line/hyperplane in the input space. However, Single Layer Perceptron is not capable of solving XOR problem, as it can implement only linearly separable logic functions by definition. Its weights vector defines a decision boundary of the form $\mathbf{w}^T \mathbf{x} + \mathbf{b} = 0$, which is a linear classifier.

5. Effect of Feature Normalization on Convergence

- Makes different features comparable by bringing their variance to the same level, so that no feature dominates the other.
- Speeds up the process of convergence.
- Makes the algorithm less sensitive to the choice of the learning rate.

11 Conclusion

In this experiment, we trained a Single Layer Perceptron from scratch on the Banknote Authentication data set to solve a binary classification task. We applied the Perceptron learning rule to update the weights and bias term of the model based on the wrongly predicted instances during training. The step activation function has been used to classify the instances into two classes. The model has successfully converged after 100 epochs, which proves that the data is linearly separable in the feature space.

12 References

1. F. Rosenblatt, “The Perceptron,” *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.
7. M. Minsky and S. Papert, *Perceptrons: An Introduction to Computational Geometry*, MIT Press, 1969.